SituationReport

get_data

Manual do Utilizador

Exportar dados Jira para SituationReport

---

situation-report -- github.com/Jaegerfeld/situation-report

Para Agile Coaches e Gestores de PI -- Version 0.51.0

1 Exportar dados do Jira Cloud

> Nota: O módulo get_data está ainda em desenvolvimento. Este manual descreve a exportação manual de dados Jira através da API REST do Jira. Este é o mesmo processo que o get_data irá automatizar — os ficheiros JSON exportados podem ser processados diretamente pelo transform_data.

Este capítulo explica como exportar dados reais de issues do Jira Cloud para o SituationReport. A exportação produz a mesma estrutura JSON que o Gerador de Dados de Teste — o transform_data processa ambos de forma idêntica.

1.1 O que precisa

Requisito	Detalhes
Conta Jira Cloud	Uma conta com acesso de leitura ao projeto pretendido
Token de API	Token pessoal de id.atlassian.com — criado uma única vez
Chave do projeto	O código abreviado do projeto Jira, ex. 'ART_A' ou 'SCRUM'
curl ou browser	curl para consultas automatizadas; browser para testes rápidos

1.2 Criar um token de API

O token de API substitui a sua palavra-passe nas chamadas de API e é criado uma única vez:

Passo	Ação
1	Abra https://id.atlassian.com no browser (com sessão iniciada no Jira)
2	Security → API tokens → Create API token
3	Introduza uma etiqueta, ex. 'SituationReport', clique em 'Create'
4	Copie o token e guarde-o em segurança — é mostrado apenas uma vez!

Nota de segurança: Trate o token de API como uma palavra-passe. Guarde-o num gestor de palavras-passe e não o partilhe.

1.3 Consultar a API REST do Jira

O Jira Cloud oferece duas variantes de API. Utilize a API v3 atual para novos scripts; os scripts v2 existentes continuam a funcionar. `expand=changelog` é obrigatório em ambas as variantes — sem ele, as transições de estado estão em falta e o transform_data não consegue calcular os valores de tempo por etapa.

API atual (v3) – recomendada

Endpoint: `POST /rest/api/3/search/jql`

Os parâmetros são passados como corpo JSON do pedido. A resposta inclui adicionalmente `isLast` e `nextPageToken` para paginação (Secção 1.5).

```
curl -X POST \
  "https://company.atlassian.net/rest/api/3/search/jql" \
  -u "name@company.com:YourAPIToken" \
  -H "Content-Type: application/json" \
  -d '{"jql":"project=ART_A ORDER BY created ASC",
       "maxResults":100,
       "expand":["changelog"],
       "fields":["issuetype","created",
                 "status","summary"]}' \
  -o ART_A_page1.json
```

maxResults está limitado a 100 issues por pedido com a API v3. Utilize nextPageToken para projetos com mais de 100 issues (Secção 1.5).

API mais antiga (v2) – ainda suportada

Endpoint: `GET /rest/api/2/search`

Parâmetros como cadeia de consulta URL. Suporta até 1.000 issues por pedido e paginação baseada em offset com `startAt`.

```
curl -u "name@company.com:YourAPIToken" \
  "https://company.atlassian.net/rest/api/2/search?\
jql=project=ART_A+ORDER+BY+created+ASC\
&expand=changelog\
&maxResults=1000\
&fields=issuetype,created,status,project,summary,resolution" \
  -o ART_A_page1.json
```

Dica: Para novos scripts, a API v3 é preferida. Os scripts v2 existentes não precisam de ser migrados.

1.4 Campos necessários para o transform_data

Campo Jira	Obrigatório	Finalidade
key	□	Identificador do issue (ex. ART_A-001)
fields.issuetype.name	□	Tipo de issue para filtragem (Feature, Bug, ...)
fields.created	□	Data de criação do issue
fields.status.name	□	Estado atual
changelog (expand=changelog)	□	Transições de estado com marcas de tempo — DEVE ser definido no URL
fields.resolution	–	Opcional — tipo de resolução (ex. 'Done', 'Won't Fix')
fields.summary	–	Opcional — título do issue

1.5 Paginação – mais de 100 issues

API v3 – Paginação baseada em cursor (nextPageToken)

A API v3 devolve no máximo 100 issues por pedido. Cada resposta contém um `nextPageToken` para a página seguinte, ou `"isLast": true` quando todos os issues foram obtidos. As páginas devem ser obtidas sequencialmente — o token da resposta atual é utilizado no pedido seguinte.

```
# Pagina 1 – sem nextPageToken
curl -X POST \
  "https://company.atlassian.net/rest/api/3/search/jql" \
  -u "name@company.com:Token" \
  -H "Content-Type: application/json" \
  -d '{"jql":"project=ART_A ORDER BY created ASC",
       "maxResults":100,"expand":["changelog"],
       "fields":["issuetype","created",
                 "status","summary"]}' \
  -o ART_A_page1.json

# Ler nextPageToken da pagina 1:
#   cat ART_A_page1.json | grep nextPageToken
#   (ou com jq: jq -r '.nextPageToken' ART_A_page1.json)

# Pagina 2 – inserir nextPageToken
curl -X POST \
  "https://company.atlassian.net/rest/api/3/search/jql" \
  -u "name@company.com:Token" \
  -H "Content-Type: application/json" \
  -d '{"jql":"project=ART_A ORDER BY created ASC",
       "maxResults":100,"expand":["changelog"],
       "fields":["issuetype","created",
                 "status","summary"],
       "nextPageToken":"<token da pagina 1>"}' \
  -o ART_A_page2.json

# Repetir ate isLast:true aparecer na resposta

# Fundir todas as paginas
python -m helper ART_A_page1.json ART_A_page2.json ... \
  --output ART_A_merged.json
```

Repita até a resposta conter `"isLast": true`. O Helper funde todas as páginas e remove duplicados automaticamente.

API v2 – Paginação baseada em offset (startAt) – Legado

A API v2 suporta até 1.000 issues por pedido. Com `startAt`, qualquer página pode ser acedida diretamente. Importante: `startAt` é compatível apenas com o endpoint v2 — não com o novo endpoint v3.

```
# Pagina 1 (issues 1-1.000)
curl -u "name@company.com:Token" \
  "https://company.atlassian.net/rest/api/2/search?\
```

```
jql=project=ART_A&expand=changelog&maxResults=1000&startAt=0" \
  -o ART_A_page1.json

# Pagina 2 (issues 1.001-2.000)
curl -u "name@company.com:Token" \
  "https://company.atlassian.net/rest/api/2/search?\
jql=project=ART_A&expand=changelog&maxResults=1000&startAt=1000" \
  -o ART_A_page2.json

# Fundir ficheiros com o Helper
python -m helper ART_A_page1.json ART_A_page2.json \
  --output ART_A_merged.json
```

Incremente `startAt` em 1.000 até a resposta conter menos de 1.000 issues.

1.6 Do exporto JSON ao ficheiro de fluxo

Após a exportação, crie um ficheiro de fluxo que mapeia os seus estados reais do Jira. O seguinte fragmento Python extrai todos os nomes de estado da exportação:

```
import json
data = json.loads(open('ART_A_page1.json').read())
statuses = set()
for issue in data['issues']:
    for h in issue['changelog']['histories']:
        for item in h['items']:
            if item['field'] == 'status':
                statuses.add(item['toString'])
print(sorted(statuses))
```

A partir dos nomes de estado encontrados, crie um `workflow.txt` — organize os estados em ordem lógica e defina os marcadores `<First>` e `<Closed>`. O formato do ficheiro de fluxo é descrito no Manual do Utilizador do transform_data.

2 Recolha automática com get_data

get_data automatiza o export do capítulo 1: obtém os issues incluindo o changelog diretamente via REST, percorre todas as páginas, remove duplicados e escreve o mesmo ficheiro JSON que o export manual produz. Ambos os caminhos de recolha são iguais — em organizações grandes, a aprovação do acesso à API pode demorar muito; até lá (e depois) o caminho manual continua de primeira classe.

```
set JIRA_TOKEN=OSeuToken
python -m get_data fetch --url https://firma.atlassian.net \
  --project ART_A --email nome@firma.pt --output ART_A.json

python -m get_data fetch --url https://jira.firma.pt \
  --project ART_A --auth bearer --api v2 --output ART_A.json

python -m get_data check ART_A_merged.json
```

A verificação (`check`) apanha os erros clássicos de export antes de o transform_data tropeçar: campos obrigatórios em falta, changelog em falta, duplicados — e páginas seguintes esquecidas (total maior do que os issues no ficheiro).

GUI: `python -m get_data` (ou o cartão Get Data no launcher) abre uma janela com os dois caminhos comutáveis: 'Jira REST fetch' e 'Existing export' (escolher ficheiro → verificar).

> *Segurança: o token nunca é guardado nem registado — a CLI lê-o da variável de ambiente (padrão JIRA_TOKEN), a GUI mantém-no apenas em memória.*

2.1 Minimização de dados na fonte (R1)

Nenhum módulo do pipeline lê um campo de pessoa - o transform_data só precisa de projeto, tipo, estado, data de criação, componentes, resolução e, do changelog, de instantes e transições de estado. Por isso a recolha REST minimiza cada export: campos por allowlist (`--keep` acrescenta campos sem referência a pessoas), o changelog fica completo, só as pessoas nele são anonimizadas (autor = marcador constante, itens de pessoa sem valores, itens de texto sem textos). Para o export manual: `python -m get_data minimise EXPORT.json` escreve EXPORT_min.json ao lado do original; `check` avisa quando há dados pessoais; botão GUI Minimizar.

> *Base jurídica (avaliação, não aconselhamento jurídico): a agregação é em si um tratamento segundo o RGPD (em regra interesse legítimo, art. 6.º n.º 1 al. f). A minimização reduz volume e risco mas não substitui a base jurídica; verifique se é necessária uma avaliação de impacto.*

3 Gerir fontes de métricas (sources)

Além dos dados brutos do Jira, o SituationReport também recolhe métricas externas (SLO/SLI, DORA, qualidade do código) — através do framework modular sources. Bastam três termos: um provider traduz um sistema estranho em records normalizados; um fetch escreve-os como registo (JSON) que a configuração da solution referencia. O relatório nunca vê um fornecedor, e os julgamentos (breached, tier low, …) são centrais — iguais para qualquer fonte.

```
python -m sources providers
```

```
python -m sources fetch --kind slo --config fontes.json --output slo.json
```

Trocar uma fonte = só o config de fetch muda (ex.: csv → prometheus no dia da aprovação da API) — mesmo registo, relatório intacto, só a coluna 'Data source' mostra a nova origem. Combinar: uma lista 'sources' enche UM registo, cada linha guarda a origem. Fonte nova = UM ficheiro em sources/providers/ com um objeto PROVIDER — descoberto automaticamente; o tutorial passo-a-passo (exemplo CSV do zero) é sources_Tutorial_EN.pdf e online em Tutorials. Tokens só de variáveis de ambiente, nunca guardados nem registados.

4 Perguntas Frequentes (FAQ)

O que acontece se expand=changelog estiver em falta?

O transform_data não encontra transições de estado e produz valores de tempo vazios para todos os issues. Os ficheiros Excel não contêm dados utilizáveis. Inclua sempre `expand=changelog` no URL.

Recebo um erro 'Unauthorized' ao chamar o curl.

Verifique: O endereço de e-mail está correto? O token de API está correto? Estão separados por dois pontos? A sua conta tem acesso de leitura ao projeto? Dica: Gere um novo token de API se o antigo foi perdido ou expirou.

Posso fundir vários projetos Jira?

Sim — seja via JQL (`jql=project in (ART_A, ART_B)`) para uma única exportação, ou fundindo duas exportações separadas com o Helper. Certifique-se de que os fluxos de trabalho de ambos os projetos são compatíveis.

Qual o tamanho máximo do ficheiro JSON?

Regra geral: aproximadamente 2–5 KB por issue (dependendo do tamanho do changelog). 1.000 issues ≈ 2–5 MB. Para projetos muito grandes (10.000+ issues), utilize paginação e o Helper.

Devo usar dados de teste ou dados reais?

Para demonstrações, formação e testes de instalação, os dados de teste sintéticos do Testdata Generator são a melhor escolha — sem preocupações de privacidade, reproduzíveis e totalmente controlados. Para análises reais de fluxo e decisões, os dados reais do Jira refletem o estado verdadeiro do fluxo de trabalho.

5 Glossário

Termo	Explicação
ART	Agile Release Train — uma estrutura de equipas em SAFe
API	Application Programming Interface — interface de programação
Token de API	Chave de segurança para aceder à API REST do Jira; substitui a palavra-passe nas chamadas curl
Changelog	Registo Jira de todas as alterações de estado de um issue (requer expand=changelog)
CFD	Cumulative Flow Diagram — mostra a contagem cumulativa de issues por etapa ao longo do tempo
Etapa Closed	A etapa que marca um issue como concluído (workflow.txt: <Closed>)
Tempo de ciclo	Tempo desde a primeira etapa ativa (<First>) até à etapa closed
Etapa First	Primeira etapa ativa onde começa a medição do tempo de ciclo (workflow.txt: <First>)
Helper	Módulo SituationReport para fundir múltiplos ficheiros JSON do Jira
Issue	Um item de trabalho no Jira (Feature, Bug, Enabler, Story, …)
JQL	Jira Query Language — linguagem de consulta para pesquisas Jira, ex. project=ART_A
JSON	JavaScript Object Notation — formato das exportações da API Jira
Etapa	Um passo no fluxo de trabalho (corresponde a uma coluna ou nome de estado Jira)
Fluxo de trabalho	Sequência ordenada de etapas pelas quais um issue passa